

Concatenando *valores*

Juntar textos e números numa frase é uma das coisas que mais se faz em programação. Nesta aula: o `+` entre textos, o erro de somar texto com número, as conversões com `str()`, as **f-strings** (o jeito moderno e recomendado), formatação de casas decimais e alinhamento, e as formas antigas que você ainda vai encontrar em código pronto.

`+` entre strings

`*` repete

`TypeError`

`str()`

f-strings

`:.2f` · `:^20`

`.format()` · `%`

`.join()`

Formação Claude Code 2026 · Baseado em docs.python.org (f-strings, Format Specification Mini-Language, str.join), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

Juntando textos *com* +

mais.py

```
nome = "Ana"
sobrenome = "Souza"
print(nome + sobrenome)           # sem espaço
print(nome + " " + sobrenome)     # com espaço
print("-" * 20)                   # repetir texto
```

SAÍDA

```
AnaSouza
Ana Souza
-----
```

- ✓ `+` entre textos **concatena**: cola um no fim do outro
- ✓ O `+` não põe espaço sozinho: inclua `" "`
- ✓ `*` com um número **repete** o texto: ótimo para linhas e separadores

O mesmo símbolo faz coisas diferentes conforme o tipo: `2 + 3` soma, `"2" + "3"` junta (`"23"`). Por isso o tipo importa tanto (aula 03).

Texto + número *não dá*

erro_tipo.py

```
idade = 29
print("Idade: " + idade)
```

```
Traceback (most recent call last):
  File "erro_tipo.py", line 2, in <module>
    print("Idade: " + idade)
          ~~~~~^~~~~~
TypeError: can only concatenate str (not "int") to str
```

- ✓ **TypeError:** a operação não existe para essa combinação de tipos
- ✓ O Python não adivinha se você quer somar ou juntar: ele pede para você decidir

str_ex.py · DUAS SAÍDAS

```
idade = 29
print("Idade: " + str(idade))    # converte para texto antes
print("Idade:", idade)          # ou o print separa com vírgula
```

```
Idade: 29
Idade: 29
```

f-strings: *o jeito moderno*

fstring.py

```
nome = "Ana"
idade = 29
preco = 49.9
print(f"{nome} tem {idade} anos.")
print(f"Ano que vem: {idade + 1}")
print(f"Preço: R$ {preco:.2f}")
print(f"{nome=}, {idade=}")    # nome e valor (bom para depurar)
```

SAÍDA

```
Ana tem 29 anos.
Ano que vem: 30
Preço: R$ 49.90
nome='Ana', idade=29
```

- ✓ Um `f` antes das aspas, e as variáveis entre `{ }`
- ✓ Converte sozinho: número entra na frase sem `str()`
- ✓ Aceita **contas** dentro das chaves: `{idade + 1}`
- ✓ `{nome=}` mostra o nome da variável e o valor: ótimo para depurar

O guia da aula 01 recomenda: **sempre use f-strings**. São as mais legíveis e as mais rápidas.

Casas decimais *e alinhamento*

DENTRO DAS CHAVES	RESULTADO	PARA
{49.9:.2f}	49.90	2 casas decimais
{0.257:.1%}	25.7%	Porcentagem
{1234567:,}	1,234,567	Separador de milhar
{"oi":>6}	" oi"	Alinhar à direita
{"oi":<6}	"oi "	Alinhar à esquerda
{"oi":^6}	" oi "	Centralizar
{7:03}	007	Zeros à esquerda

A regra é `{valor:formato}` . Os dois-pontos separam o que mostrar de como mostrar. O mesmo formato funciona no `.format()` .

O separador de milhar do Python é a **vírgula** e o decimal é o **ponto**, no padrão americano. Para "1.234,56" no padrão brasileiro, troque os caracteres depois de formatar, ou use o módulo `locale` .

```
valor = f"{1234.5:,.2f}"    # '1,234.50'
br = valor.replace(",","X").replace(".",",").replace("X",".")
print(br)                  # 1.234,50
```

.format(), % e .join()

outros.py

```
nome, idade = "Ana", 29
print("{} tem {} anos.".format(nome, idade))  # .format()
print("%s tem %d anos." % (nome, idade))      # estilo antigo
print(", ".join(["maçã", "banana", "uva"]))   # junta uma lista
```

SAÍDA

```
Ana tem 29 anos.
Ana tem 29 anos.
maçã, banana, uva
```

FORMA	QUANDO APARECE
<code>f"..."</code>	Código novo: prefira esta
<code>"...".format()</code>	Código de antes do Python 3.6; modelos de texto guardados
<code>"%s" % valor</code>	Código antigo; mensagens do módulo logging
<code>", ".join(lista)</code>	Juntar muitos textos com um separador

`.join()` só aceita textos: uma lista de números precisa ser convertida antes.

Montando um texto *aos poucos*

acumular.py

```
mensagem = "Olá"  
mensagem += ", Ana"  
mensagem += "!"  
print(mensagem)
```

SAÍDA

```
Olá, Ana!
```

- ✓ `+=` junta ao que já existe: `mensagem = mensagem + ", Ana"`
- ✓ Útil para montar um texto em várias etapas
- ✓ Para juntar centenas de pedaços, guarde numa lista e use `.join()` no fim: é bem mais rápido

Qual usar *em cada caso*

SITUAÇÃO	MELHOR ESCOLHA	EXEMPLO
Frase com variáveis	f-string	f"{nome} tem {idade} anos"
Dinheiro, medidas	f-string com formato	f"R\$ {preco:.2f}"
Só mostrar na tela	print com vírgulas	print("Idade:", idade)
Juntar dois textos	+	nome + " " + sobrenome
Juntar uma lista de textos	.join()	", ".join(frutas)
Linha separadora	*	"_" * 30

Com o agente: peça *"reescreva este print usando f-string e explique a diferença"* num código antigo. Ver o antes e o depois ensina rápido.

Um *recibo* formatado

Enunciado: imprima um recibo com título centralizado entre linhas de `=`, cliente, produto, total com 2 casas e o valor de cada parcela.

Uma resposta possível

```
# Exercício: recibo com f-strings
cliente = "Ana Souza"
produto = "Curso de Python"
valor = 197
parcelas = 3
print("=" * 32)
print(f"{'RECIBO':^32}")
print("=" * 32)
print(f"Cliente: {cliente}")
print(f"Produto: {produto}")
print(f"Total:    R$ {valor:.2f}")
print(f"{parcelas}x de R$ {valor / parcelas:.2f}")
```

SAÍDA

```
=====
                        RECIBO
=====
Cliente: Ana Souza
Produto: Curso de Python
Total:    R$ 197.00
3x de R$ 65.67
```

- ✓ `{'RECIBO':^32}` centraliza em 32 posições
- ✓ A conta da parcela fica dentro da f-string
- ✓ Desafio: mostre o total no padrão brasileiro, com vírgula

Teste o que *ficou*

1. O que "Idade: " + 29 causa?

A) Idade: 29

B) TypeError

C) ValueError

D) Idade: + 29

B. Converta com str() ou use f-string.

2. Qual mostra 7.50?

A) f"{7.5:.2f}"

B) f"{7.5:2}"

C) str(7.5)

D) f"{7.5}"

A. .2f = duas casas decimais.

3. Quanto vale "ab" * 3?

A) "ab3"

B) "ababab"

C) Erro

D) 6

B. * repete o texto.

4. Para juntar ["a", "b", "c"] como "a-b-c":

A) "-".join(lista)

B) lista.join("-")

C) lista + "-"

D) str(lista)

A. O separador chama o join.

Frases montadas *do jeito certo*

`+` junta textos e `*` repete; texto com número dá **TypeError**. As **f-strings** resolvem quase tudo, com contas e formatos como `:.2f` e `:^20`. `.format()` e `%` existem em código antigo, e `.join()` junta listas de textos.

`+` `·` `*`

`TypeError`

`f"{valor:.2f}"`

`{nome=}`

`.join()`

Próxima aula → 06 · Utilizando Operadores

Fontes: docs.python.org/3/reference/lexical_analysis.html#f-strings; docs.python.org/3/library/string.html#format-specification-mini-language; docs.python.org/3/library/stdtypes.html (`str.join`, `str.format`, `printf-style`), consultados em outubro de 2026. Exemplos em `exemplos/`, executados com Python 3.14.4.